FUZZY-LOGIC-BASED QOS CONTROL IN ONEM2M

ADAPTIVE EDGE-TO-CLOUD TRAFFIC OFFLOADING

RAJ MANOJ ASHUTOSH JOHN AYUSH

PAPER BY ZYANE ABDELLAH & ET-TOUSY JAMAL, FICLOUD 2026
PRESENTED AT ABV-IIITM GWALIOR

PAPER PRESENTATION 2026

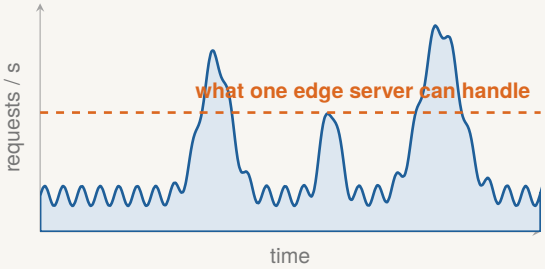
THE PLAN FOR THE NEXT FEW MINUTES

01	02	03	04	05
Raj	**Manoj**	**Ashutosh**	**John**	**Ayush**
Why this paper?	What is the system?	How does the brain decide?	Did it work?	Is it any good, really?

One question runs through all five parts:

When an IoT server gets busy, how much work should it hand to the cloud?

IOT TRAFFIC DOES NOT ARRIVE POLITELY



- Billions of devices: factories, hospitals, smart cities, wearables.
- Each sends tiny messages (10–50 KB), but **all at once** when something happens.
- Load changes **without warning**: devices move, apps change, resources are shared.

The result

Above the dashed line: requests queue, latency explodes, some requests fail.

WHAT “QUALITY OF SERVICE” MEANS HERE

RTT
Round-trip time
How long a request takes to come back

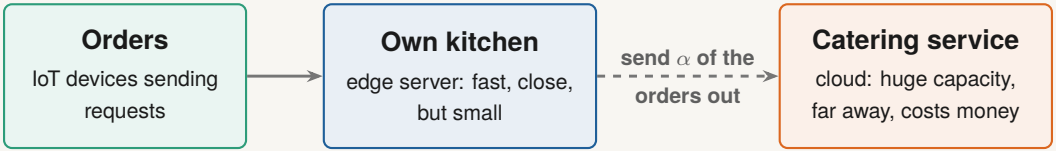
% OK
Success rate
Share of requests that do not fail

CPU
CPU usage
How hard the server is working

RAM
RAM usage
How full its memory is

QoS = keep latency bounded, throughput sufficient, service available.
The paper measures it with exactly these four numbers. Hold on to them.

AN ANALOGY: A BUSY RESTAURANT KITCHEN



Send nothing out
kitchen drowns, orders are late or lost

Send everything out
you pay for catering you did not need

The paper's idea
choose the share α continuously, like a head chef

PROBLEM STATEMENT: FIXED RULES BREAK WHEN THE LOAD MOVES

How it is usually done

Static thresholds: *"if CPU $>$ 70 %, push traffic to the cloud."*

Classification, scaling and NFV-based models tuned once, offline.

Why that fails

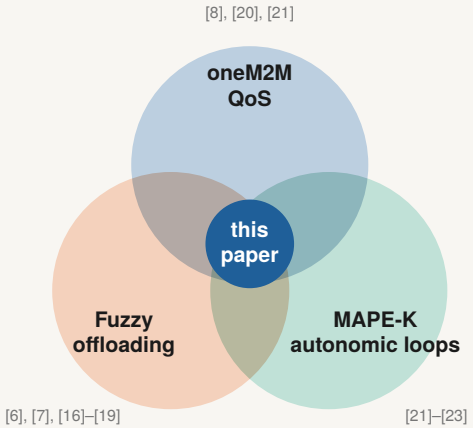
IoT load is bursty and uncertain. A hard line either reacts too late, or flips back and forth around the threshold.

Who pays the price

- Degraded service: higher RTT, failed requests
- Higher operating cost: cloud over-provisioned
- Latency-critical apps hurt most: remote healthcare, industrial automation

Needed: a controller that adapts at run time, without an exact model of the system.

THE RESEARCH GAP: THREE IDEAS THAT HAD NOT MET



- oneM2M QoS studies exist, but use **fixed** traffic rules.
- Fuzzy offloading exists, but **outside** oneM2M middleware.
- MAPE-K loops exist for oneM2M, but without fuzzy reasoning.

Gap: a fuzzy controller living *inside* oneM2M, deciding offloading in real time.

RELATED WORK AT A GLANCE

QoS frameworks for IoT	Edge / fog offloading	Fuzzy logic for offloading	oneM2M QoS + MAPE-K
Good monitoring and management, but slow to adapt and heavy to run	Heuristics, optimisation and ML; most need exact parameter predictions	Handles uncertainty with IF–THEN rules; FI-RBTOM, fuzzy Q-learning	Authors' own earlier work: protocol QoS, SLA-driven automatic traffic control
[8]–[10]	[11]–[15]	[6], [7], [16]–[19]	[20], [21]

Every column solves part of the problem. None of them does all of it inside the oneM2M standard.

WHAT THE PAPER CONTRIBUTES

1. A **fuzzy model** built for oneM2M QoS: 3 inputs, 27 rules, 1 output.
2. Placed **inside a MAPE-K self-management loop**, so it re-decides every 5 s.
3. **Tested** on a real oneM2M server (Mobius) with an AWS cloud tier.

Coming up: Manoj shows the system, Ashutosh opens up the controller, John shows the results, Ayush puts them in perspective.

headline result at peak load

43.5%
lower cost

55.9%
lower round-trip time

+19.4%
more requests succeed

02

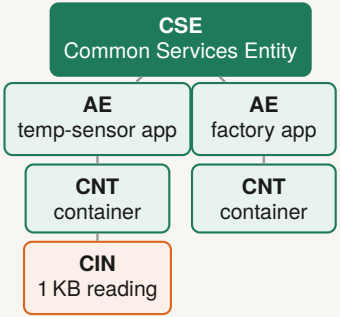
MANOJ

Background and proposed system architecture

oneM2M in one slide • the three-layer architecture (Fig. 1) • the step-by-step workflow (Fig. 2) • edge vs. cloud • the Autonomic QoS Manager and its MAPE-K loop

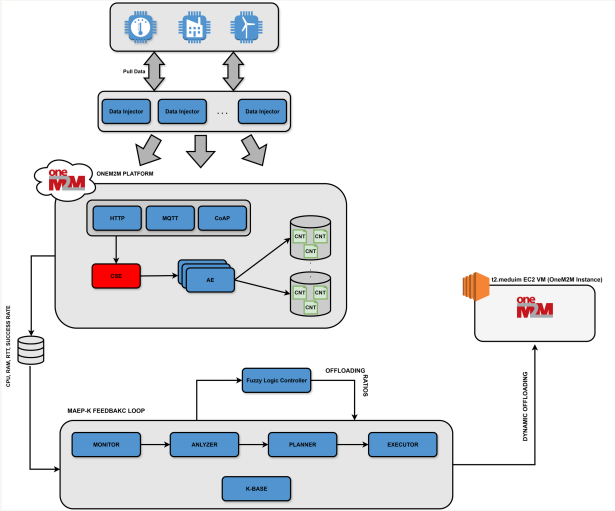
ONEM2M: ONE COMMON LANGUAGE FOR IOT

- A global **standard middleware** (service layer) between devices and applications.
- Vendor- and protocol-neutral: the same REST resources over **HTTP, MQTT, CoAP**.
- Everything is a **resource** in a tree, created and read with CREATE / RETRIEVE …
- **Mobius**: an open-source oneM2M server, used in this paper.



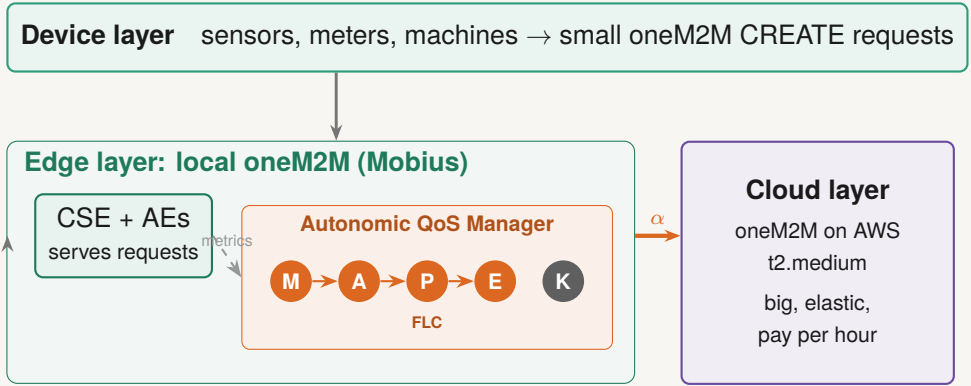
Mca: app ↔ CSE Mcc: CSE ↔ CSE (e.g. edge ↔ cloud)

THE PROPOSED ARCHITECTURE (PAPER, FIG. 1)



1. **Devices** (sensors, factory, wind turbine) feed **data injectors**.
2. The **edge oneM2M platform**: protocol bindings → CSE → AEs → containers.
3. A **database** stores CPU / RAM / RTT / success rate.
4. The **MAPE-K loop** with the **Fuzzy Logic Controller** watches it all.
5. Offloaded traffic goes to a second **oneM2M instance on AWS**.

SAME PICTURE, THREE LAYERS

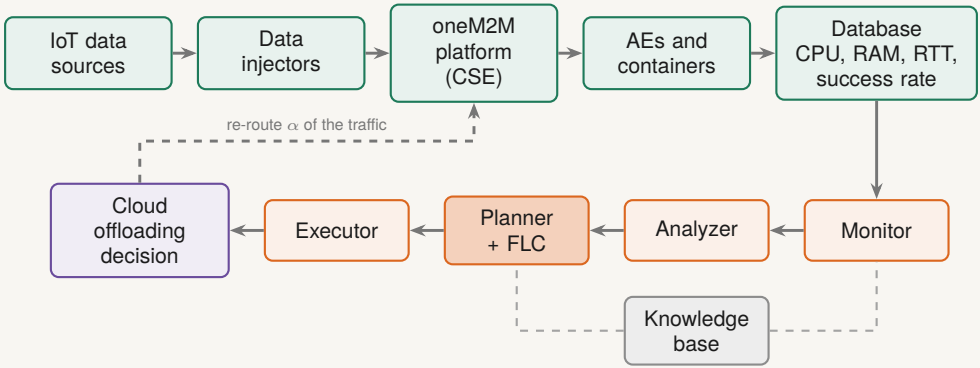


$1 - \alpha$ of the traffic stays at the edge, α goes to the cloud. Only α is decided.

EDGE LAYER VS. CLOUD LAYER

	Edge	Cloud
Where	Gateways, small servers next to devices	Data centres, far away
Strength	Low delay, no cloud bill	Lots of CPU, storage, networking
Weakness	Small: saturates under bursts	Extra latency and a per-hour cost
Best for	Time-sensitive, light work	Heavy work, overflow during peaks
In the paper	Ubuntu 22.04, 4 vCPU, 4 GiB + Mobius	AWS EC2 t2.medium + Mobius

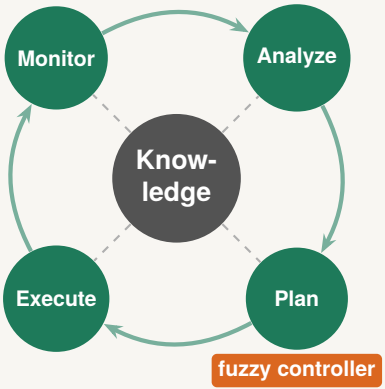
HOW A REQUEST AND A DECISION FLOW (PAPER, FIG. 2)



Top row: the data path. Bottom row: the control loop that watches it and changes the routing.

THE AUTONOMIC QOS MANAGER: A MAPE-K LOOP

one turn every 5 s



Monitor Collect CPU, RAM, RTT, queue length, bandwidth from the platform and edge nodes.

Analyze Clean and normalise the numbers; spot coming overload.

Plan The **Fuzzy Logic Controller** turns the state into an offloading share α.

Execute Re-route that share of traffic to the cloud CSE.

Knowledge Shared store: policies, rules, history of past decisions.

HAND-OFF: OPEN THE “PLAN” BOX



Everything so far is plumbing. The interesting part is **how α is chosen**.

Over to Ashutosh.

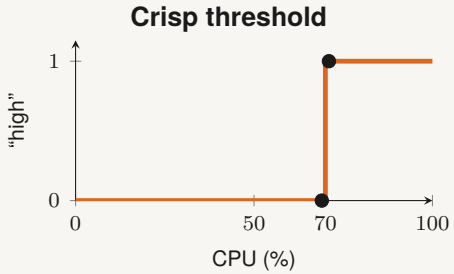
03

ASHUTOSH

The fuzzy logic controller and methodology

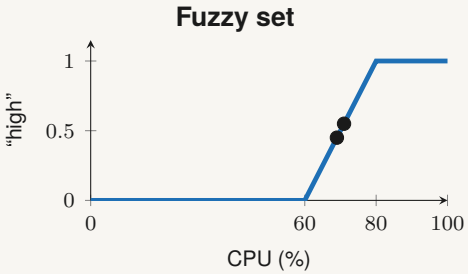
why fuzzy • inputs and output $\alpha(t)$ • membership functions • the 27-rule base (Table I) • inference and defuzzification • a worked example • monotonicity principles • testbed and controller configuration

WHY FUZZY? IS 69 % CPU “HIGH”?



69 % = “not high”, 71 % = “high”.

Two nearly equal states, opposite actions.

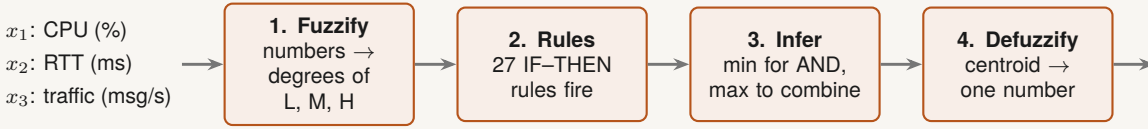


69 % is 0.45 high, 71 % is 0.55 high.

Small change in, small change out.

Fuzzy logic turns expert sentences (“if CPU is high and RTT is high, offload a lot”) into a smooth controller, **no exact system model needed**.

THE CONTROLLER IN ONE LINE



Output

$\alpha(t) \in [0, 1]$: share of incoming traffic sent from the local oneM2M platform to the cloud.

$\alpha = 0$: all local. $\alpha = 1$: all to cloud.

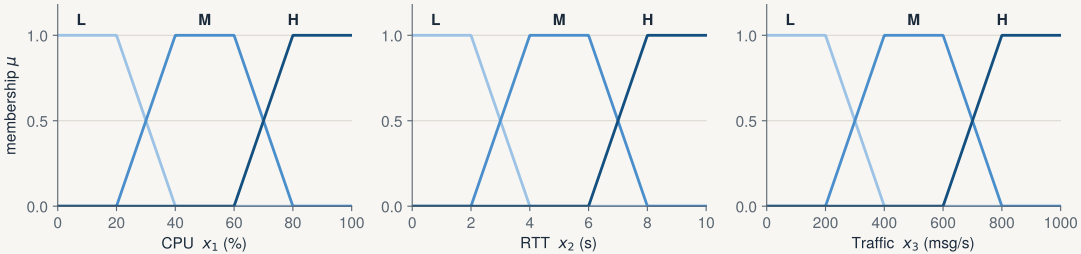
Ranges used

CPU $\in [0, 100]$ %

RTT $\in [0, T_{\max}]$, $T_{\max} = 10$ s

Traffic $\in [0, R_{\max}]$, $R_{\max} = 1000$ msg/s

STEP 1: MEMBERSHIP FUNCTIONS

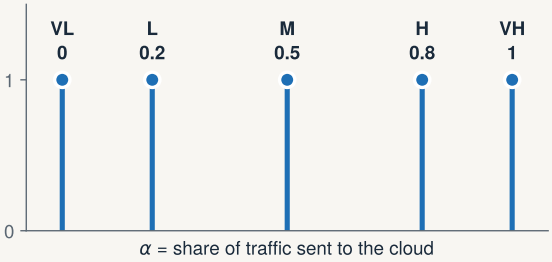


$$\mathrm{trap}(x; a, b, c, d) = \max\Big(\min\Big(\tfrac{x-a}{b-a},\ 1,\ \tfrac{d-x}{d-c}\Big), 0\Big)$$

Breakpoints at 0, 20, 40, 60, 80, 100 % of each range, e.g.
$\mu_{\text{CPU-M}}(x_1) = \mathrm{trap}(x_1; 20, 40, 60, 80)$

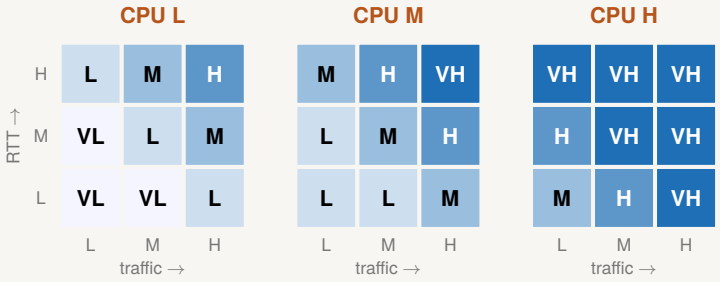
Uniform partition, then light hand-tuning of $T_{\max}$, $R_{\max}$ and the high-end thresholds on held-out calibration traces. Neighbouring sets overlap, so a value can be partly M and partly H.

THE OUTPUT: FIVE LEVELS OF OFFLOADING



- Five terms: Very Low, Low, Medium, High, Very High.
- Each is a **singleton**: a spike at one value.
  $\{0.0,\ 0.2,\ 0.5,\ 0.8,\ 1.0\}$
- Inputs are Mamdani-style fuzzy sets, outputs are Sugeno-style spikes: cheap to defuzzify.

STEP 2: THE 27-RULE BASE (TABLE I)



$3 \times 3 \times 3 = 27$ rules, every combination covered.

Each rule reads:
IF CPU is A_k *AND RTT is* B_k *AND traffic is* C_k *THEN* α *is* D_k

Written from expert knowledge of oneM2M QoS behaviour. Darker = offload more. It only gets darker as you go **up**, **right**, or to the next **CPU** grid.

STEP 3 AND 4: INFERENCE AND DEFUZZIFICATION

Firing strength of rule k (Zadeh AND)

$$w_k(t) = \min\left(\mu_{A_k}(x_1),\ \mu_{B_k}(x_2),\ \mu_{C_k}(x_3)\right)$$

A rule is only as true as its weakest condition.

Combine rules with the same output (max)

$\mu_C(\text{VH}) = \max$ of all w_k whose rule says VH, and so on for each level.

Centroid defuzzification

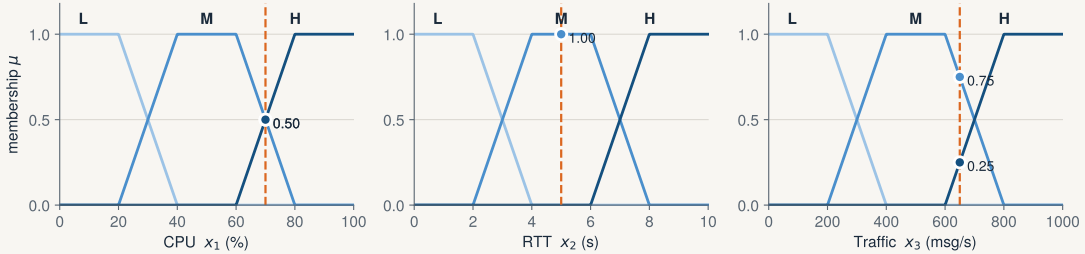
$$\alpha(t) = \frac{\sum_i x_i\, \mu_C(x_i)}{\sum_i \mu_C(x_i)}$$

$x_i \in \{0, 0.2, 0.5, 0.8, 1\}$: a weighted average of the five spikes.

Costs < 0.5 ms per cycle on a t2.medium: the controller is never the bottleneck.

WORKED EXAMPLE (1/2): FUZZIFY

Edge server state: **CPU = 70 %**, **RTT = 5 s**, **traffic = 650 msg/s**.



	L	M	H
CPU 70 %	0	**0.50**	**0.50**
RTT 5 s	0	**1.00**	0
Traffic 650	0	**0.75**	**0.25**

Computed with our re-implementation of the controller, `scripts/flc.py`.

WORKED EXAMPLE (2/2): RULES, COMBINE, DEFUZZIFY

Combine (max): $\text{M} = 0.50$, $\text{H} = 0.25$, $\text{VH} = 0.50$

rule	IF CPU, RTT, traffic	w_k
14	M, M, M $\rightarrow$ **M**	$\min(.5, 1, .75) = 0.50$
15	M, M, H $\rightarrow$ **H**	$\min(.5, 1, .25) = 0.25$
23	H, M, M $\rightarrow$ **VH**	$\min(.5, 1, .75) = 0.50$
24	H, M, H $\rightarrow$ **VH**	$\min(.5, 1, .25) = 0.25$

Other 23 rules: strength 0.

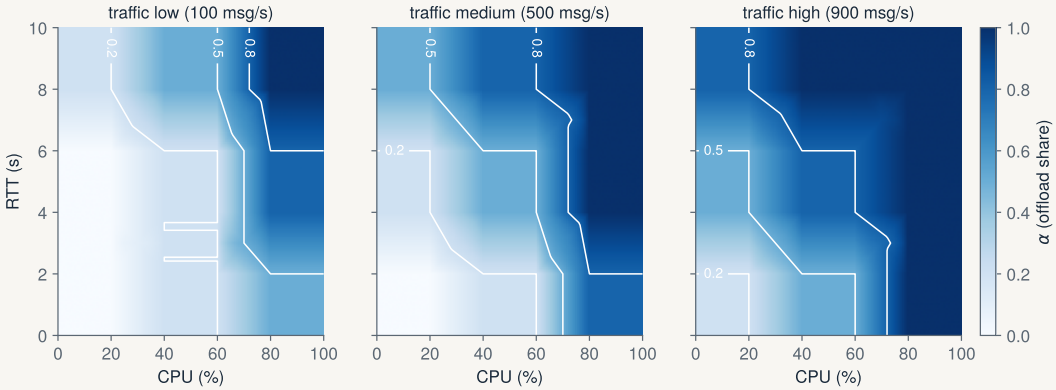
$$\alpha = \frac{0.5 \cdot 0.50 + 0.8 \cdot 0.25 + 1.0 \cdot 0.50}{0.50 + 0.25 + 0.50}$$

$$\alpha = \frac{0.95}{1.25} = \mathbf{0.76}$$

Meaning

Send about **3 of every 4** requests to the cloud right now.

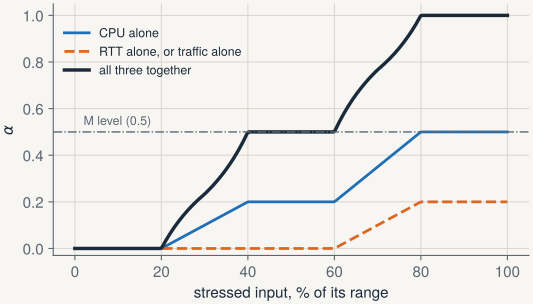
THE WHOLE CONTROLLER AS A PICTURE



Darker = more offloading. The surface rises smoothly with every input: no cliffs, unlike a threshold.

Our plot of the paper's rule base and membership functions (`scripts/make_figs.py`).

THREE DESIGN PRINCIPLES BUILT INTO THE RULES



1. **Monotone**: more CPU, RTT or traffic never means less offloading.
2. **No panic on one spike**: one input saturated alone stays at or below M (0.5).
   Robust to a single noisy sensor.
3. **All three high** $\Rightarrow \alpha = 1$: protect the edge node.

Curves: one input swept from 0 to 100 % of its range, the others held low.

TESTBED AND CONTROLLER CONFIGURATION

Testbed

- **Edge**: Ubuntu 22.04, 4 vCPU, 4 GiB; Mobius CSE + QoS Manager + FLC + Knowledge base.
- **Cloud**: Mobius CSE on AWS EC2 t2.medium.
- Interfaces: oneM2M Mca / Mcc over HTTP and MQTT.
- Monitor: CPU, RAM every 1 s from Linux `/proc`; RTT from timestamps added by the proxy.
- Load: JMeter (HTTP) + Eclipse Paho (MQTT) virtual devices.

Controller settings

MAPE-K period	5 s (< 3 % of a core)
Inference	Mamdani min–max
Defuzzification	centroid
Input smoothing	EMA, 3 samples, $\beta = 0.6$
Hysteresis	act only if $\lvert\Delta\alpha\rvert \geq 0.05$
$T_{\max}$, $R_{\max}$	10 s, 1000 msg/s

Only the period, the smoothing and the hysteresis were tuned; the rules and breakpoints stayed fixed.

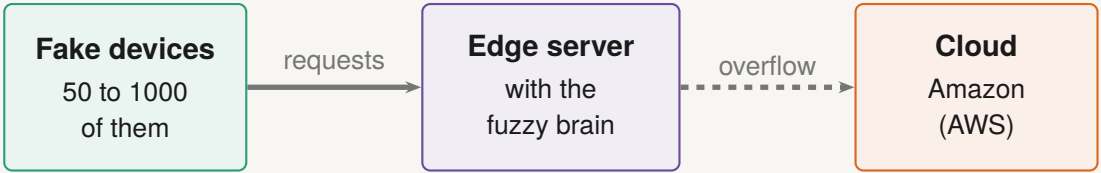
04

JOHN

Did it work?

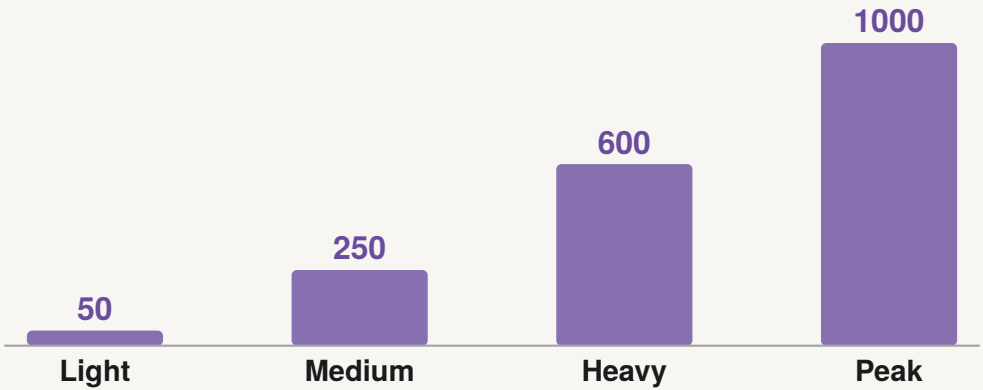
the test in one picture • four load levels • three scenarios • what happened to speed, reliability and CPU • what happened to the bill

THE TEST IN ONE PICTURE



Each run: **30 minutes**, repeated **5 times**.

FOUR LEVELS OF LOAD



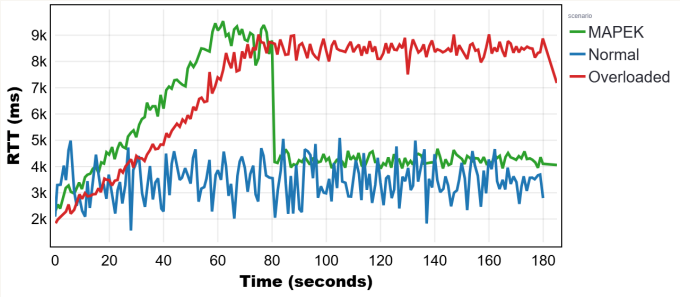
devices, each sending about **one request per second**

THREE SCENARIOS

Normal	Overloaded	With fuzzy
normal traffic, no help needed	heavy traffic, no help	heavy traffic, fuzzy brain ON

Same heavy traffic in the last two. **Only difference: the fuzzy brain.**

SPEED: RESPONSE TIME OVER TIME



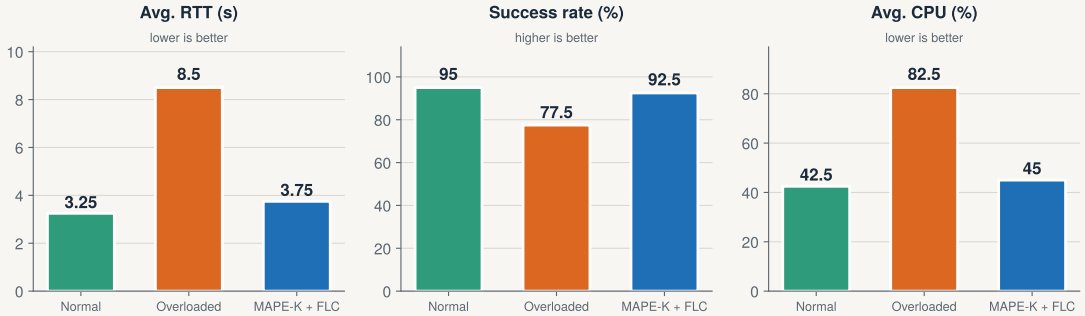
Paper, Fig. 3(a)

Blue: normal.

Red: overloaded, gets slow and stays slow.

Green: fuzzy brain kicks in at about 80 s and brings it back down.

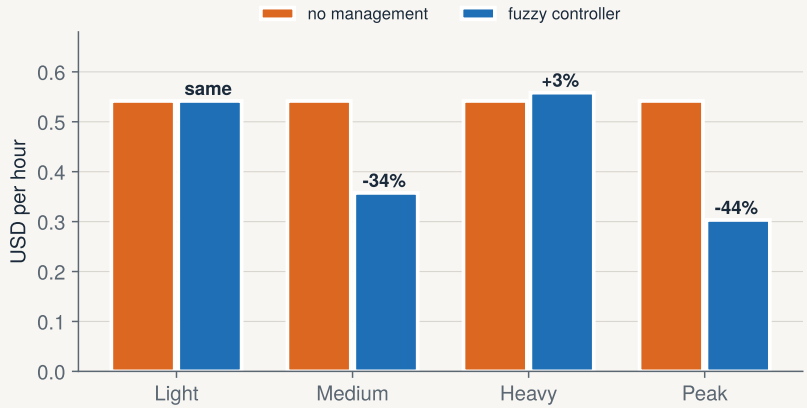
ALL THREE SCENARIOS, SIDE BY SIDE



Overloaded is bad on everything. **With fuzzy** is almost back to normal.

Values from the paper's Table III; chart redrawn by us.

THE BILL: CLOUD COST PER HOUR



Saves money at **medium** and **peak** load.

Heavy costs slightly more on purpose: it spends a little to keep service good.

Values from the paper's Table III; chart redrawn by us.

THREE NUMBERS TO REMEMBER

43.5%
cheaper

55.9%
faster

19.4%
more requests succeed

compared with doing nothing, under heavy load.

Is that good enough? Over to Ayush.

05

AYUSH

Comparison, critical analysis and conclusion

baselines • static threshold vs. fuzzy (Fig. 4) • Table IV • checking the numbers • our own re-simulation • scalability • strengths and limitations • summary and future work

WHAT IT IS COMPARED AGAINST

1. Unmanaged

Everything processed locally. Same as the Overloaded scenario: a default Mobius install.

Worst case.

2. Static threshold

Offload **100 %** when CPU $>$ 70 %; back to local when CPU $<$ 50 %.

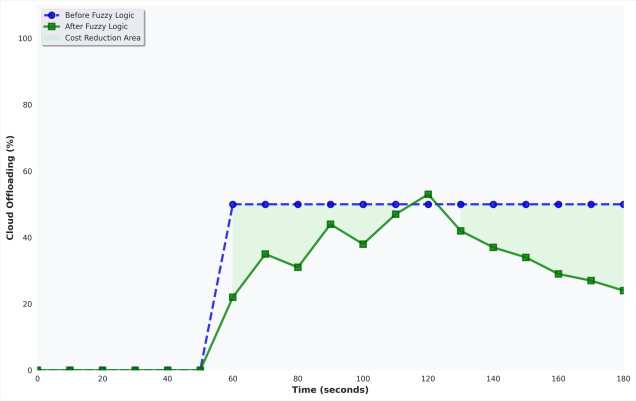
Typical industry “cloud bursting”.

3. Literature

FI-RBTOM (fuzzy rules) [6], fuzzy Q-learning [7], load-balance-aware collaborative offloading [11].

Compared qualitatively only.

STATIC THRESHOLD VS. FUZZY: HOW α MOVES (PAPER, FIG. 4)



- **Static** (dashed): 0 % until the CPU limit, then jumps to a fixed rate and stays there.
- **Fuzzy** (solid): rises with the peak, then **backs off** as load falls.
- Shaded area = cloud capacity the static rule pays for but does not need.

TABLE IV, SIMPLIFIED

	Unmanaged	Static	FI-RBTOM	Fuzzy QL	Collab.	**Proposed**
RTT	8.5 s	better	better	better	better	**3.75 s**
Success rate	77.5 %	better	better	better	better	**92.5 %**
Cloud cost	$2.17	over-prov.	n/r	lower	lower	**$1.22**
Oscillation	none	high	moderate	low	low	**low**
Inside oneM2M	native	external	external	external	external	**native**
Needs training	no	no	no	yes	no	**no**

Only the **Unmanaged** and **Proposed** columns are measured on this testbed.
The rest is the direction of improvement reported in each cited paper.

CHECKING THE HEADLINE NUMBERS

	Overloaded	Fuzzy	change
Total cost	\$2.1667	\$1.2242	−43.5 % ✓
RTT	8.50 s	3.75 s	−55.9 % ✓
Success	77.5 %	92.5 %	+19.4 % ✓

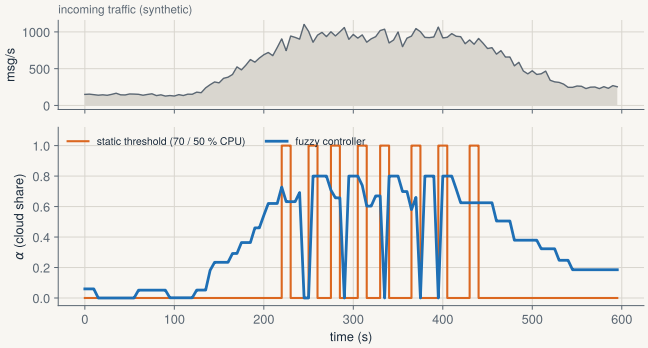
All three reproduce from Table III.
But: +19.4 % is *relative*; in absolute terms it is **+15 percentage points**.

Where the cost comes from

	before	after
AWS runtime	\$0.1667	\$0.0942
Operational	\$2.0000	\$1.1300
Total	**\$2.1667**	**\$1.2242**

t2.medium: 2 vCPU, 4 GiB, \$0.0464/h on demand.
\$0.1667 ≈ 3.6 instance-hours (our arithmetic).
The “operational” cost is 92 % of the total but is not defined in the paper.

WE RE-IMPLEMENTED THE CONTROLLER: WHAT A CLOSED LOOP DOES



Toy edge server, same rules, EMA and hysteresis as the paper.

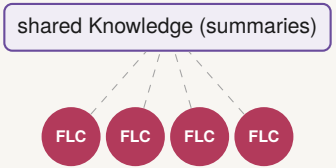
- **Static**: slams between 0 and 1: the bang-bang the paper warns about.
- **Fuzzy**: graded, follows the load up and back down.
- But it also **hunts**: offloading lowers CPU and RTT, which then lowers α.

The paper does not analyse loop stability.

Illustration only: synthetic plant with a first-order lag, not the paper's data. Code: `scripts/make_figs.py`.

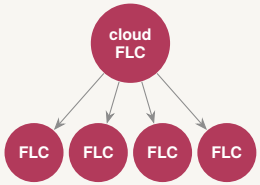
SCALING TO HUNDREDS OF EDGE NODES

Per-node managers



Fully decentralised. Traffic $O(N)$, decision latency constant.

Hierarchical FLC



Cloud-level FLC sets per-region offload caps; local FLCs stay independent.

27 rules + centroid < 0.5 ms per 5 s cycle per node. **Argued, not yet measured:** the testbed has one edge node and one cloud VM.

CRITICAL ANALYSIS

Strengths

- Fully **interpretable**: 27 readable rules, no training data.
- Runs **inside** oneM2M (Mobius): any oneM2M app benefits unchanged.
- Cheap: < 0.5 ms per decision.
- Clear, sensible design principles (monotone, no single-spike panic).
- Improves QoS *and* cost at the same time.

Limitations we found

- **One edge node, one VM, synthetic traffic**; scalability is argued, not measured.
- Literature comparison is **qualitative**; the static baseline has no numbers.
- Means of 5 runs, **no variance**; CPU and RAM averages **identical** in every row of Table III.
- "Operational" cost (92 % of the bill) is never defined.
- Text promises bandwidth and task size as inputs; the controller uses CPU, RTT, traffic.
- Hand-tuned breakpoints; **loop stability** not studied (see our re-sim).

SUMMARY AND FUTURE WORK

Summary

- A 27-rule fuzzy controller inside a MAPE-K loop decides **how much** oneM2M traffic to send to the cloud.
- Under overload: RTT −55.9 %, success +19.4 %, cost −43.5 %, CPU back to 40–50 %.
- A practical, low-cost option for oneM2M platforms; the evidence is still small-scale.

Future work (from the paper)

1. Auto-tune breakpoints and rule weights (RL or evolutionary search).
2. Multi-objective control: QoS + energy + cost.
3. Security-aware QoS: DDoS and anomaly signals as fuzzy inputs.
4. Distributed / hierarchical MAPE-K on a real multi-host edge testbed.

THANK YOU

Questions?

01 Raj | 02 Manoj | 03 Ashutosh | 04 John | 05 Ayush

Zyane A., Et-Tousy J., “Fuzzy-Logic-Based QoS Control in oneM2M: Adaptive Edge-to-Cloud Traffic Offloading,” FiCloud 2026, pp. 214–222, doi:10.1109/FiCloud70576.2026.00036